

COMP3026E Distributed Systems

4. Distributed Snapshots

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen

Distributed Snapshots



- What is the state of a distributed system?



- N processes in the system with no process failures
 - Each process has some state it keeps track of
- There are two first-in, first-out, unidirectional channels between every process pair P and Q
 - Call them $\text{channel}(P, Q)$ and $\text{channel}(Q, P)$
 - The channel has state, too: the set of messages inside
 - All messages sent on channels arrive intact, unduplicated, in order



Aside: FIFO Communication Channel



All messages sent on channels arrive intact, unduplicated, in order

- Q: Arrive?



Aside: FIFO Communication Channel



All messages sent on channels arrive intact, unduplicated, in order

- Q: Arrive?
- Q: Intact?



Aside: FIFO Communication Channel



All messages sent on channels arrive intact, unduplicated, in order

- Q: Arrive?
- Q: Intact?
- Q: Unduplicated?



Aside: FIFO Communication Channel



All messages sent on channels arrive intact, unduplicated, in order

- Q: Arrive?
- Q: Intact?
- Q: Unduplicated?
- Q: In order?
- Q: How can we get such a FIFO channel?

Aside: FIFO Communication Channel



All messages sent on channels arrive intact, unduplicated, in order

- Q: Arrive?
- Q: Intact?
- Q: Unduplicated?
- Q: In order?
- Q: How can we get such a FIFO channel?
 - A: At-least-once retransmission + at-most-once deduplication + ordering of messages (seq no + delivery in seq no order)

TCP provides all of these when processes don't fail

Global Snapshot is Global State



- Each distributed application has a number of processes running on a number of physical servers
- These processes communicate with each other via channels
- A **global snapshot** captures
 - The local states of each process (e.g., program variables), and
 - The state of each communication channel

Why do We Need Snapshots?



1. **Checkpointing**: Restart if the application fails

Why do We Need Snapshots?



1. **Checkpointing**: Restart if the application fails
2. **Collecting garbage**: Remove objects that aren't referenced

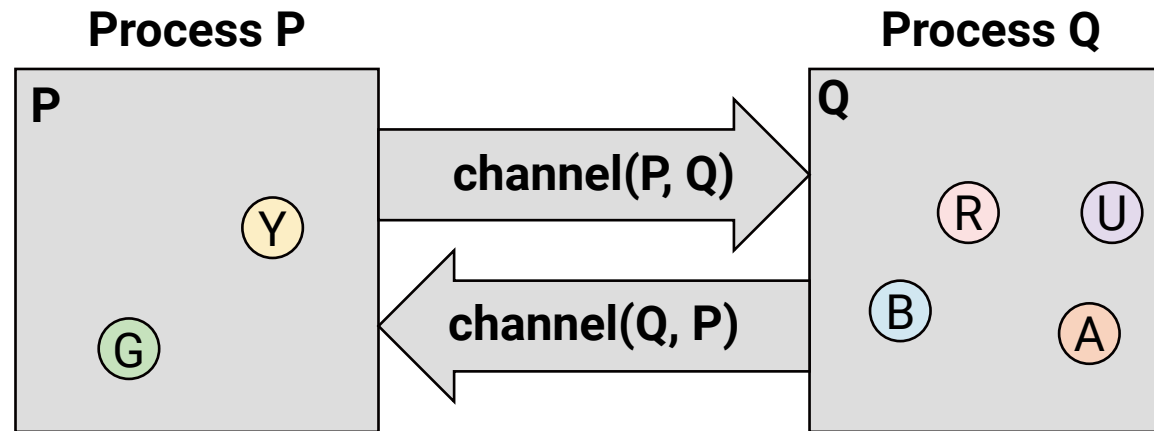
Why do We Need Snapshots?

1. **Checkpointing**: Restart if the application fails
2. **Collecting garbage**: Remove objects that aren't referenced
3. **Detecting deadlocks**: The snapshot can examine the current application state
 - Process A grabs Lock 1, B grabs Lock 2, A waits for Lock 2, B waits for Lock 1...

System Model: Graphical Example

- Let's represent process state as a set of colored tokens
- Suppose there are two processes, P and Q :

Objective Correct global snapshot = Exactly one of each token

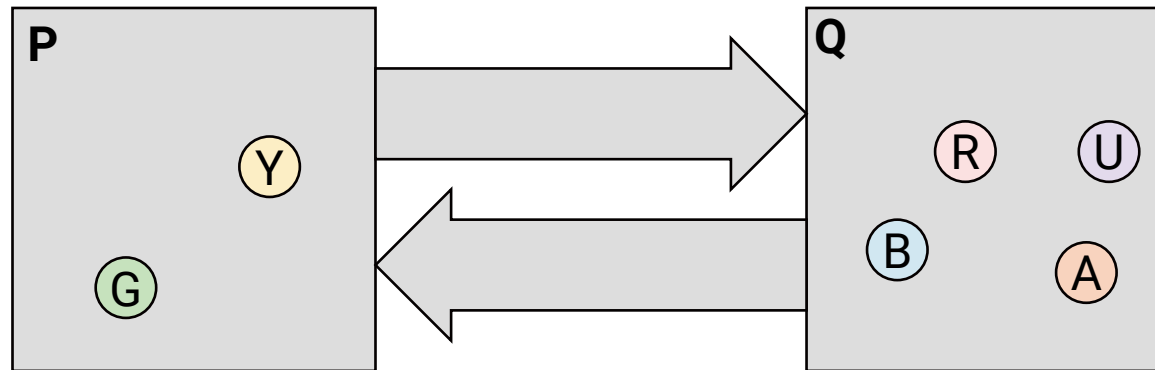


When is Inconsistency Possible?

- Suppose we take snapshots only from a process perspective
- Suppose snapshots happen independently at each process
- Let's look at the implications...

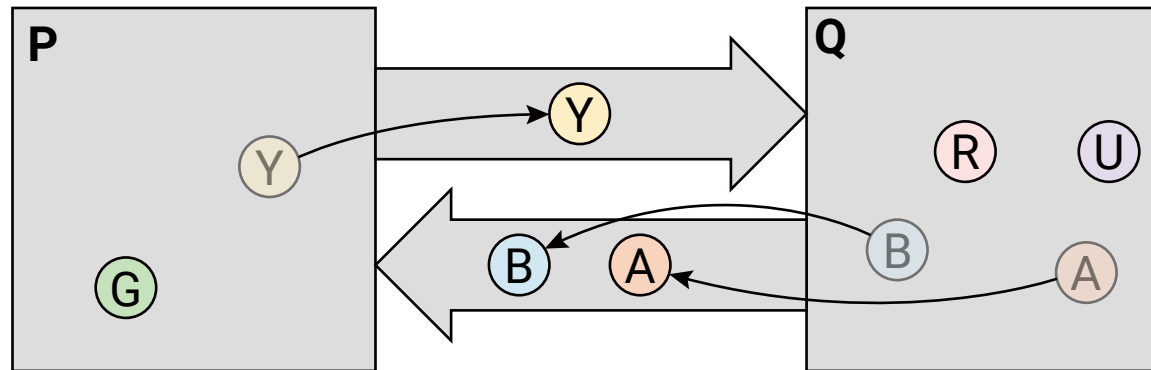
Problem: Disappearing Tokens

- P, Q put tokens into channels, then snapshot



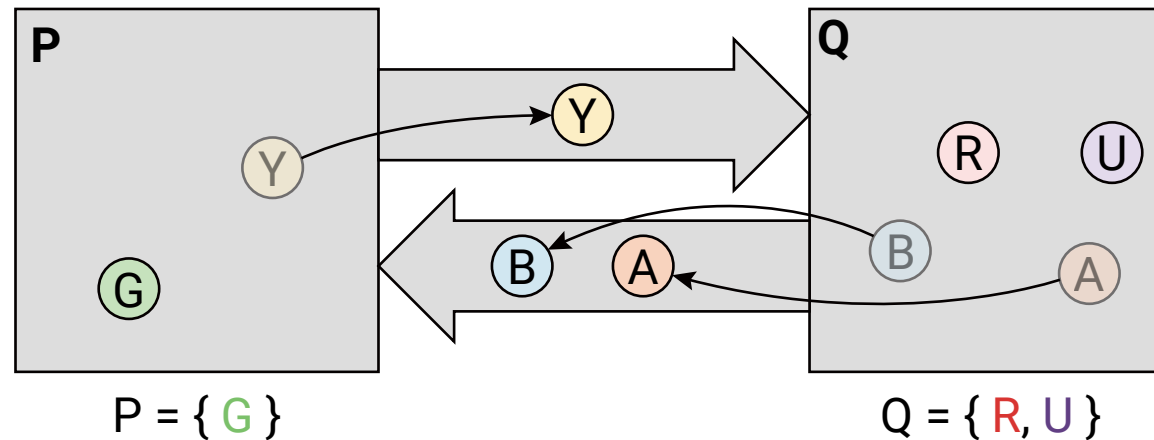
Problem: Disappearing Tokens

- P, Q put tokens into channels, then snapshot



Problem: Disappearing Tokens

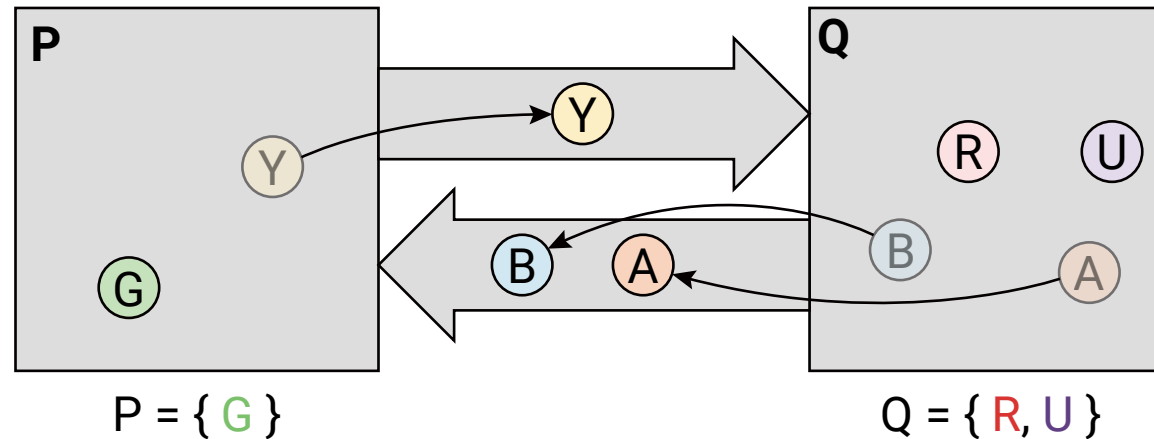
- P, Q put tokens into channels, then snapshot



Problem: Disappearing Tokens

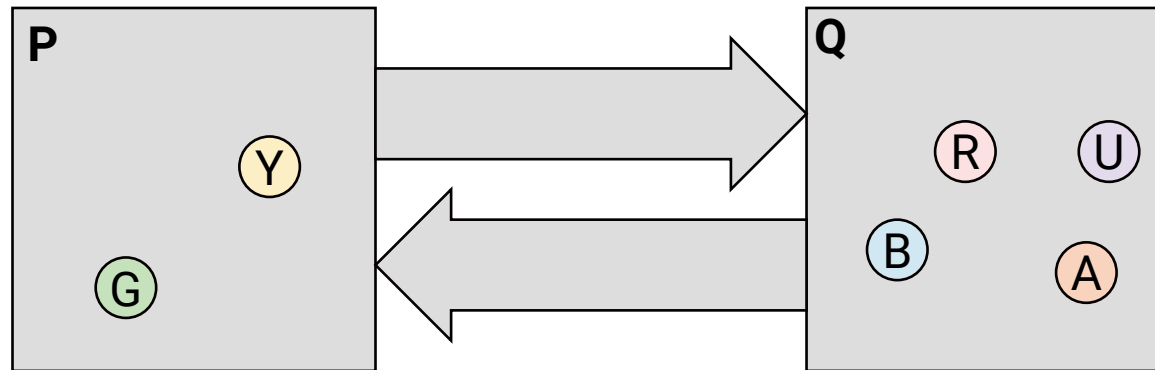
- P, Q put tokens into channels, then snapshot

This snapshot **misses** Y, B, and A tokens



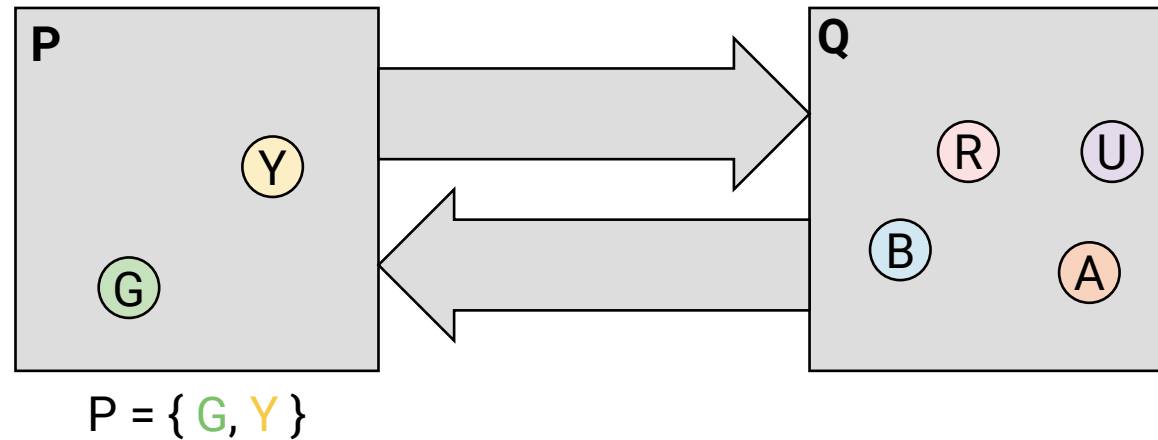
Problem: Duplicated Tokens

- P snapshots, then sends Y



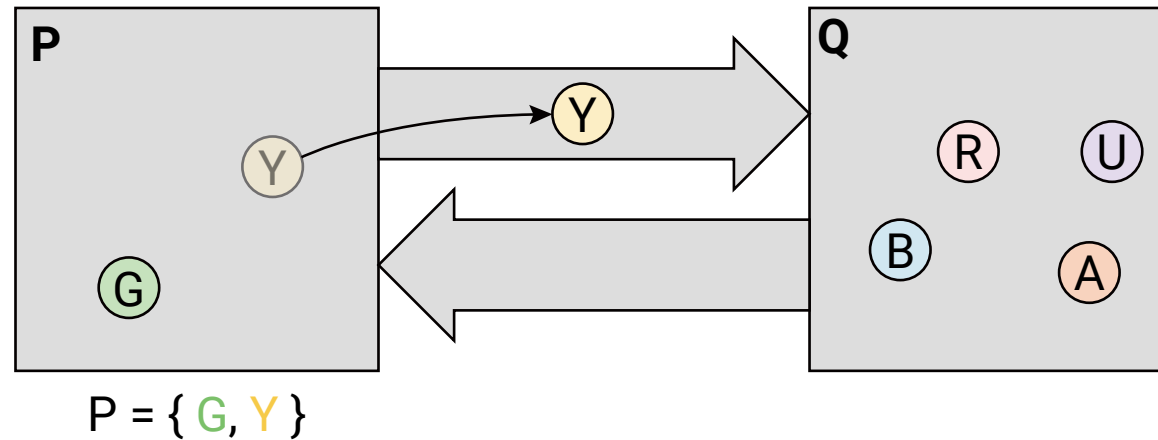
Problem: Duplicated Tokens

- P snapshots, then sends Y



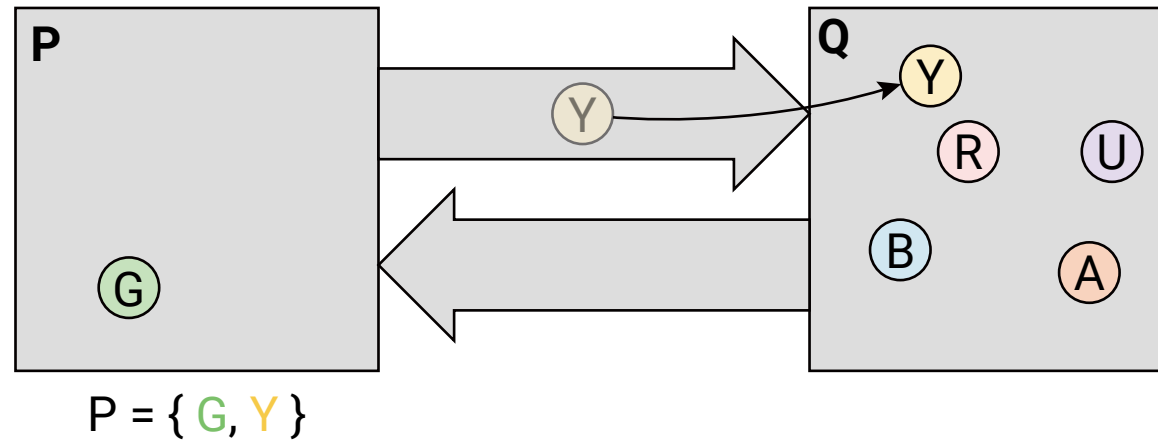
Problem: Duplicated Tokens

- P snapshots, then sends Y



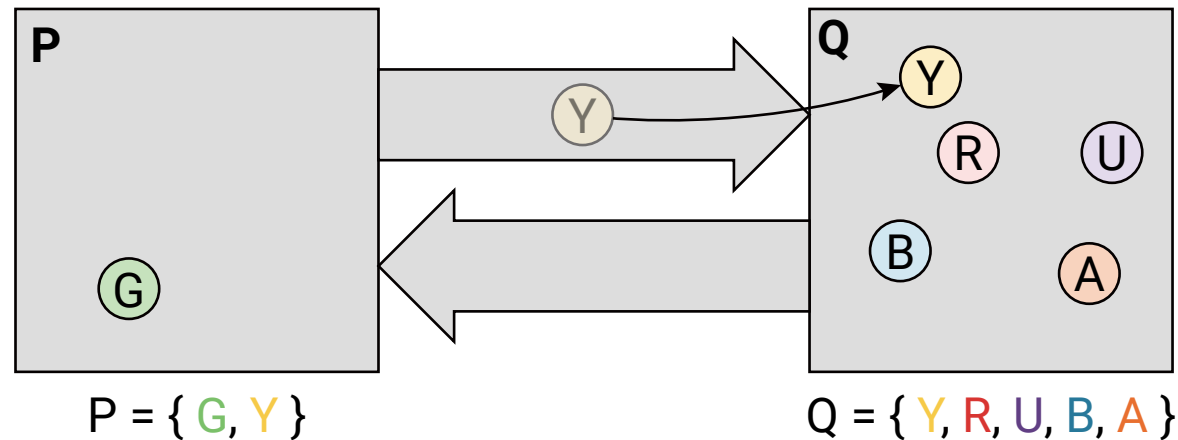
Problem: Duplicated Tokens

- P snapshots, then sends Y



Problem: Duplicated Tokens

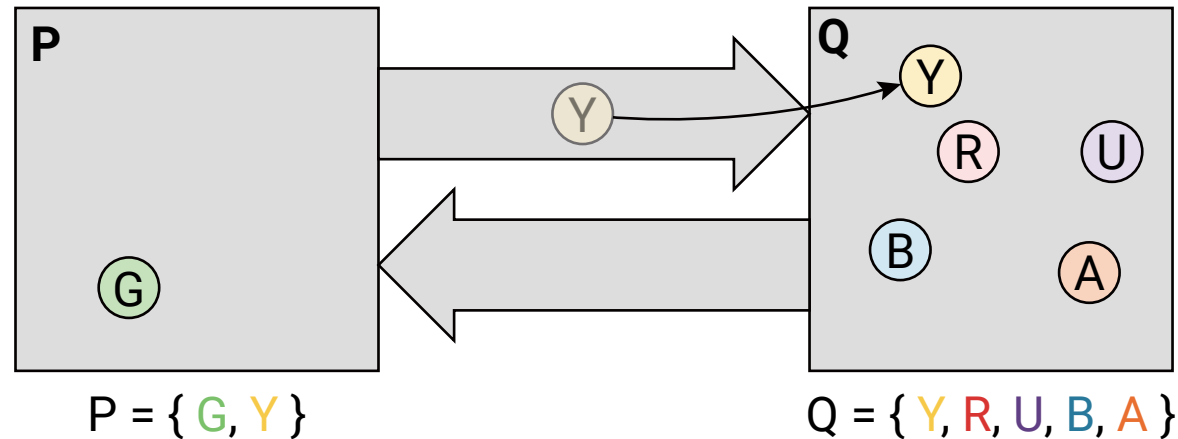
- P snapshots, then sends Y
- Q receives Y , then snapshots



Problem: Duplicated Tokens

- P snapshots, then sends Y
- Q receives Y , then snapshots

This snapshot **duplicates** the Y token



Idea: “Marker” Messages



- What went wrong?
 - We should have captured the state of the **channels** as well
 - Avoids missing tokens
 - We should have coordinated snapshots at different processes
 - Avoids duplicate tokens
- Let's send a **marker message** ▲
 - Used to capture channel state and coordinate different processes
 - Distinct from other messages
 - Channels deliver marker and other messages FIFO

Chandy-Lamport Algorithm: Overview

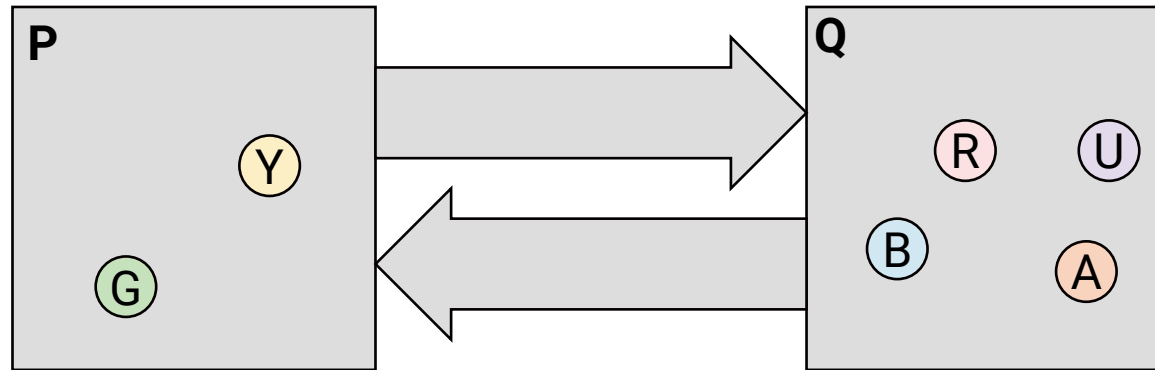
- We'll designate one node (say P) to start the snapshot
 - Without any steps in between, P :
 1. Records its local state ("snapshots")
 2. Sends a marker on each outbound channel

Chandy-Lamport Algorithm: Overview

- We'll designate one node (say P) to start the snapshot
 - Without any steps in between, P :
 1. Records its local state ("snapshots")
 2. Sends a marker on each outbound channel
- Nodes remember whether they have snapshotted
- On receiving a marker, a non-snapshotted node performs steps 1 and 2 above

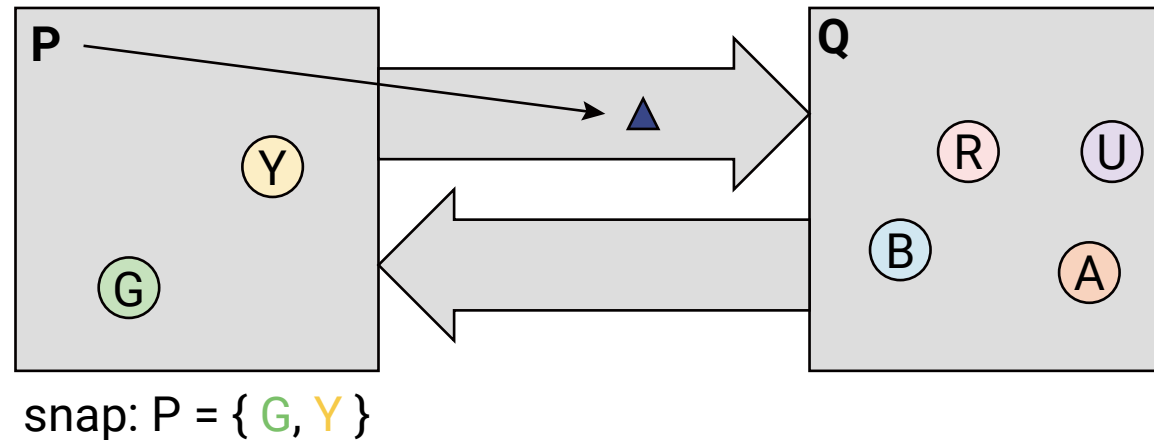
Chandy-Lamport: Sending Process

- P snapshots and sends marker, then sends Y



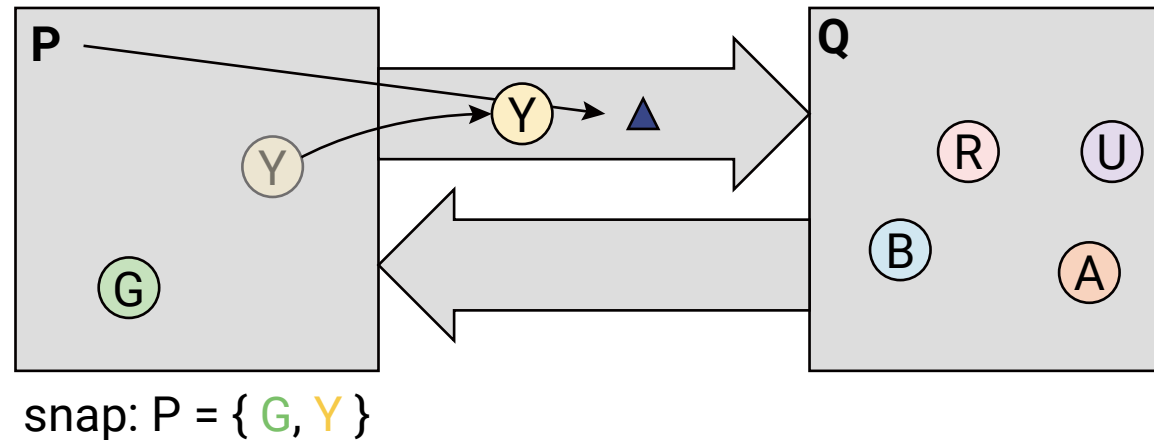
Chandy-Lamport: Sending Process

- P snapshots and sends marker, then sends Y



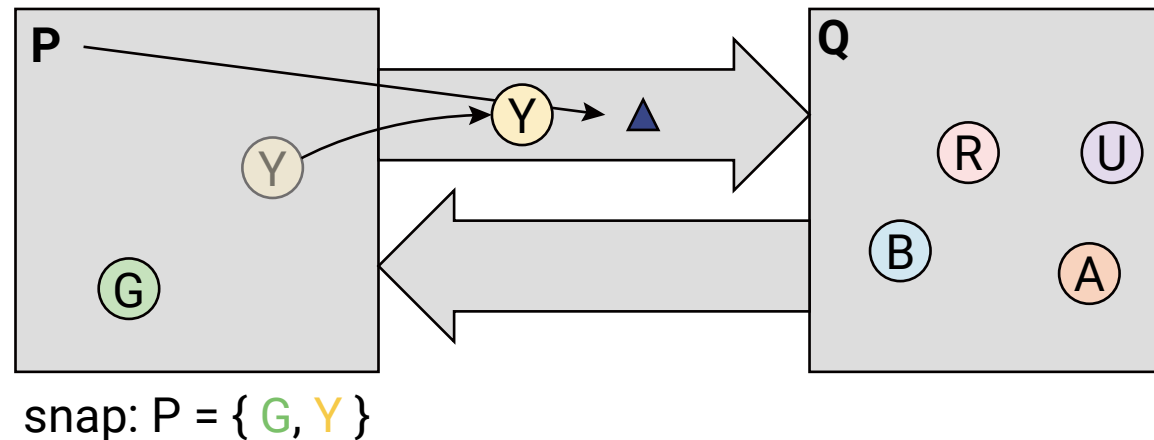
Chandy-Lamport: Sending Process

- P snapshots and sends marker, then sends Y



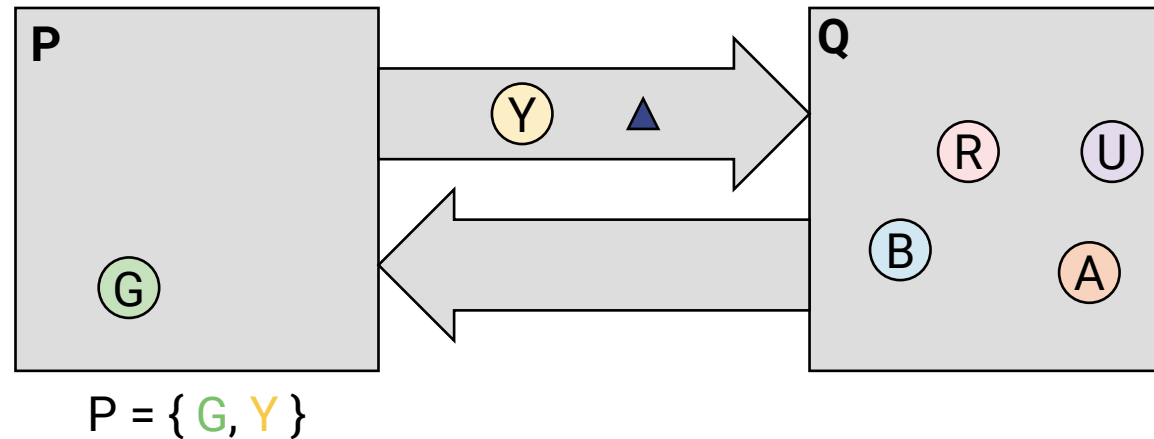
Chandy-Lamport: Sending Process

- P snapshots and sends marker, then sends Y
- **Send Rule:** Send marker on all outgoing channels
 - Immediately after snapshot
 - Before sending any further messages



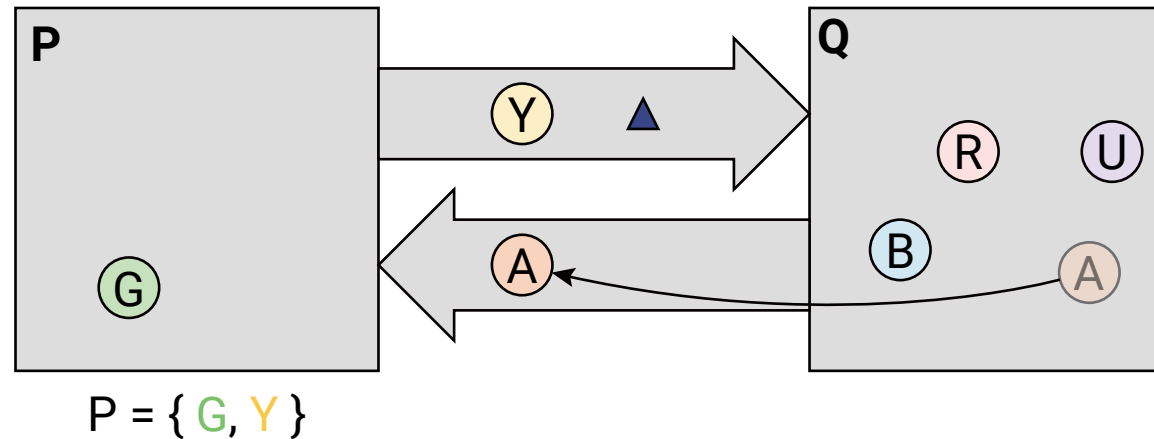
Chandy-Lamport: Receiving Process

- At the same time, Q sends orange token O



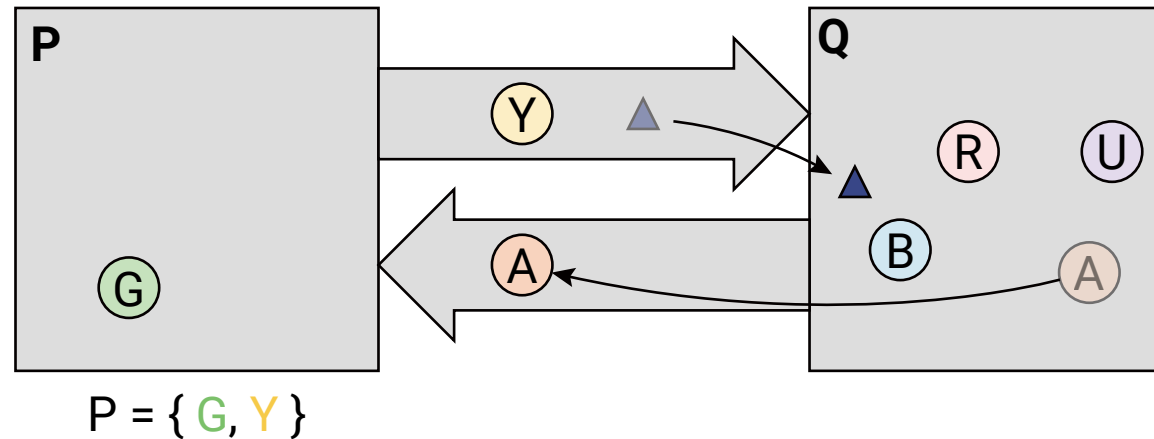
Chandy-Lamport: Receiving Process

- At the same time, Q sends orange token O



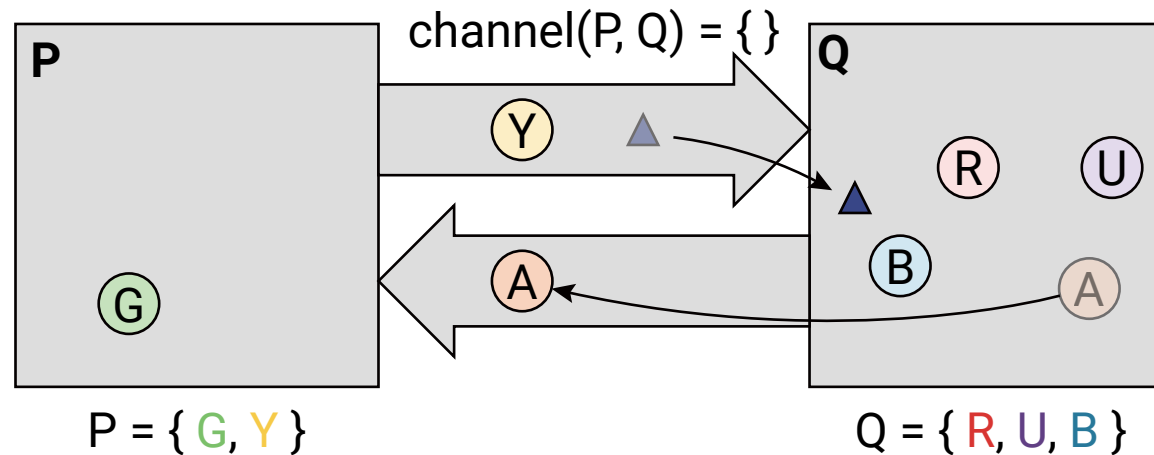
Chandy-Lamport: Receiving Process

- At the same time, Q sends orange token O
- Then, Q receives marker $\blacktriangle$



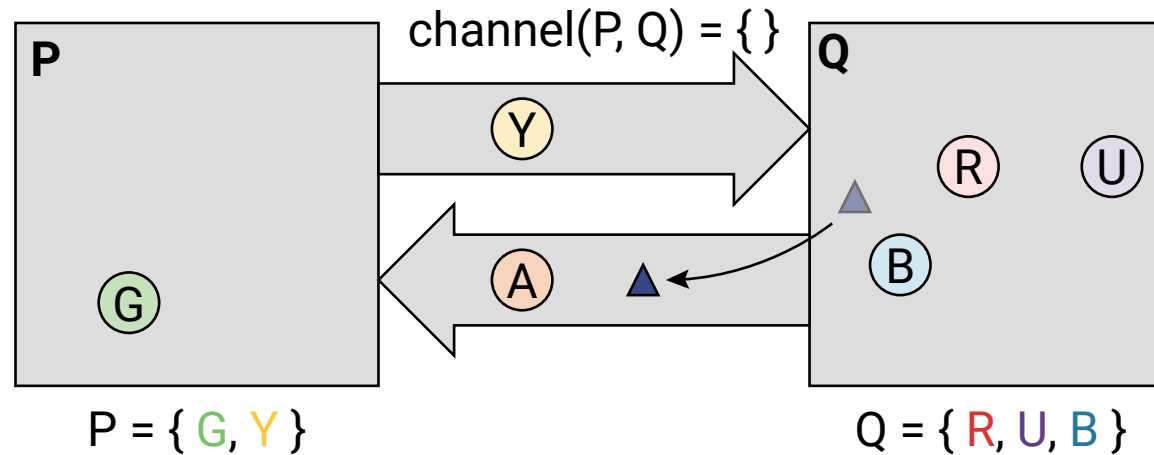
Chandy-Lamport: Receiving Process

- At the same time, Q sends orange token 0
- Then, Q receives marker $\blacktriangle$
- **Receive Rule (if not yet snapshotted)**
 - On receiving marker on channel c record c 's state as empty



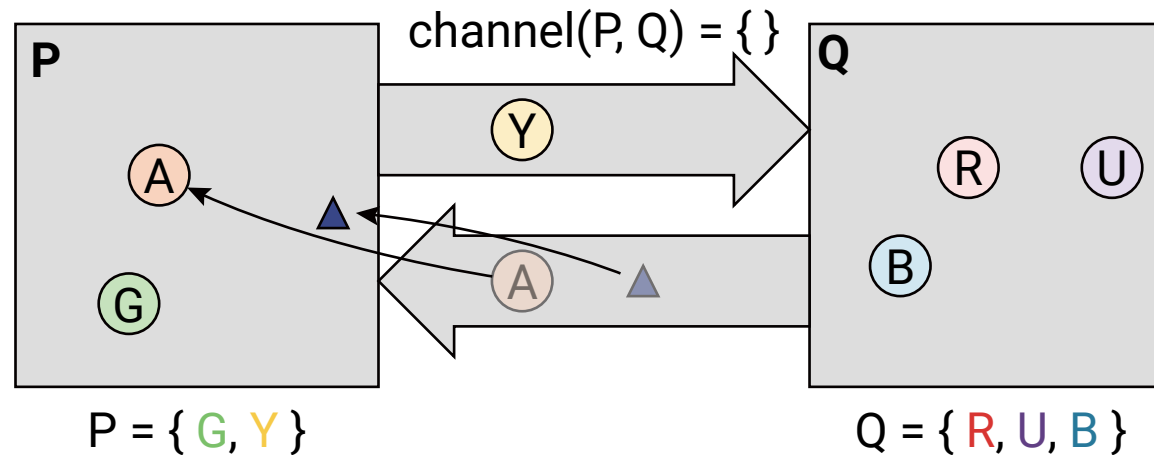
Chandy-Lamport: Receiving Process

- Q sends marker to P



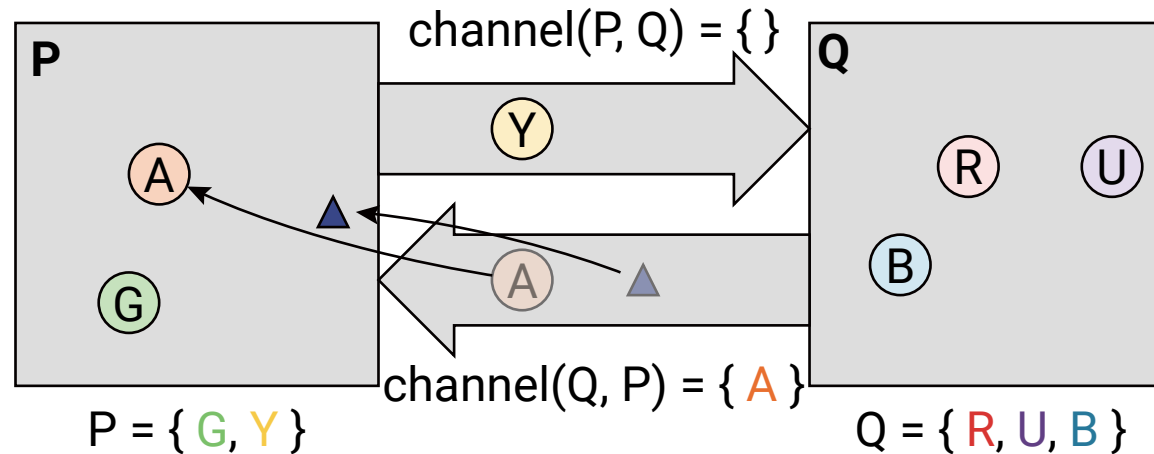
Chandy-Lamport: Receiving Process

- Q sends marker to P
- P receives orange token $\circ$, then marker $\blacktriangle$



Chandy-Lamport: Receiving Process

- Q sends marker to P
- P receives orange token 0, then marker ▲
- **Receive Rule (if already snapshotted)**
 - On receiving marker on channel c record c 's state: all messages from c since snapshot



Chandy-Lamport Algorithm: Details

- We'll designate one node (say P) to start the snapshot
 - Without any steps in between, P :
 1. Records its local state ("snapshots")
 2. Sends a marker on each outbound channel
 3. Starts recording messages on each inbound channel

Chandy-Lamport Algorithm: Details

- We'll designate one node (say P) to start the snapshot
 - Without any steps in between, P :
 1. Records its local state ("snapshots")
 2. Sends a marker on each outbound channel
 3. Starts recording messages on each inbound channel
- Nodes remember whether they have snapshotted

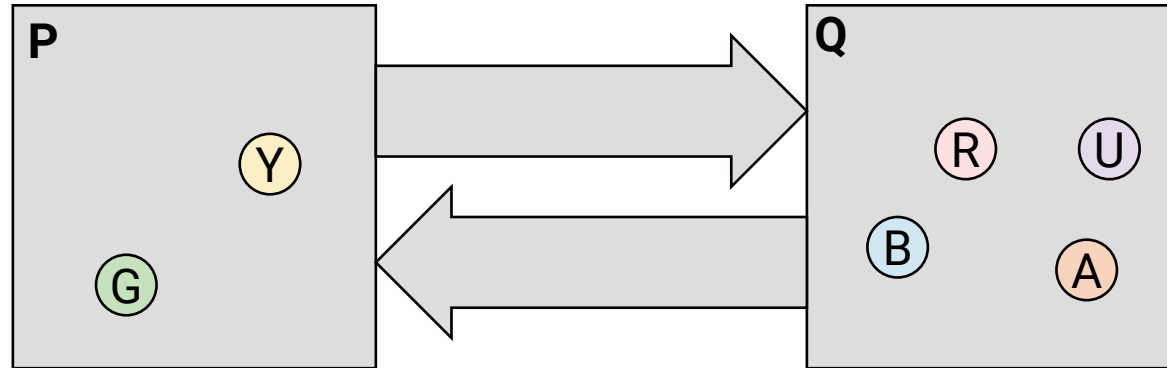
Chandy-Lamport Algorithm: Details

- We'll designate one node (say P) to start the snapshot
 - Without any steps in between, P :
 1. Records its local state ("snapshots")
 2. Sends a marker on each outbound channel
 3. Starts recording messages on each inbound channel
- Nodes remember whether they have snapshotted
- On receiving a marker on channel c
 - If ($\neg$ snapshotted) then performs steps 1–3
 - 4. Stop recording messages on channel c

Chandy-Lamport Puzzles #1

- Is this snapshot possible? And if so, how?

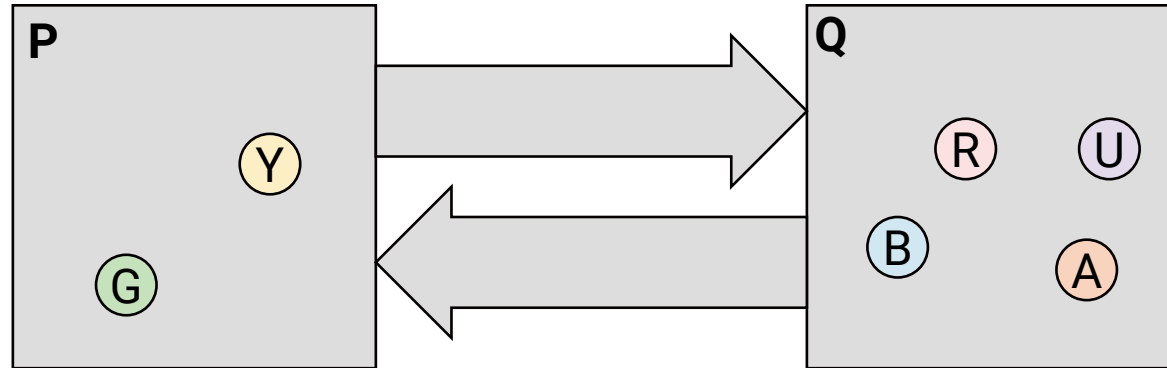
$P = \{ G \}$
 $ch(P, Q) = \{ Y \}$
 $Q = \{ R, U \}$
 $ch(Q, P) = \{ B, A \}$



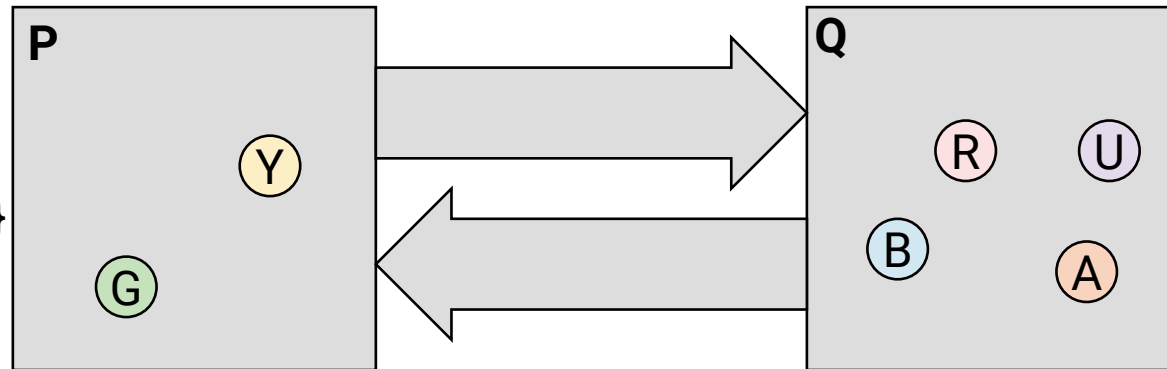
Chandy-Lamport Puzzles #2

- Is this snapshot possible? And if so, how?

$P = \{ G, Y, R, U, B, A \}$
 $ch(P, Q) = \{ \}$
 $Q = \{ \}$
 $ch(Q, P) = \{ \}$



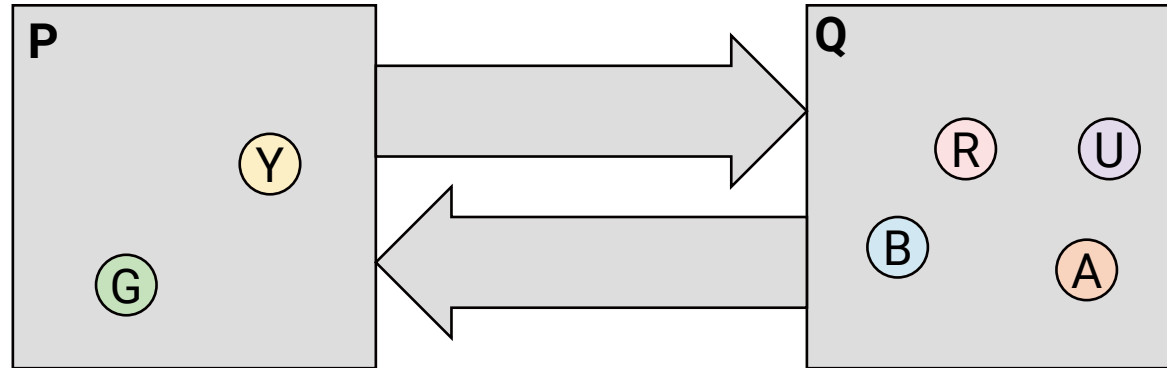
$P = \{ \}$
 $ch(P, Q) = \{ \}$
 $Q = \{ \}$
 $ch(Q, P) = \{ G, Y, R, U, B, A \}$



Chandy-Lamport Puzzles #3

- Is this snapshot possible? And if so, how?

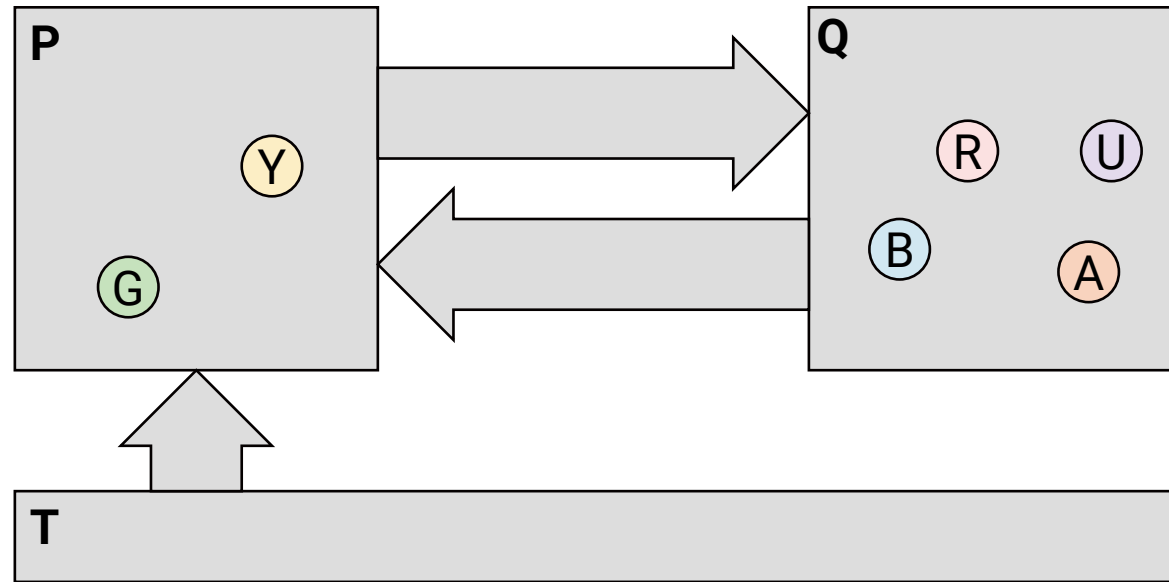
$P = \{ G, Y \}$
 $ch(P, Q) = \{ R \}$
 $Q = \{ B, A \}$
 $ch(Q, P) = \{ U \}$



Chandy-Lamport Puzzles #4

- Is this snapshot possible? And if so, how?

$P = \{ G, Y \}$
 $ch(P, Q) = \{ R \}$
 $Q = \{ B, A \}$
 $ch(Q, P) = \{ U \}$
 $T = \{ \}$
 $ch(T, P) = \{ \}$



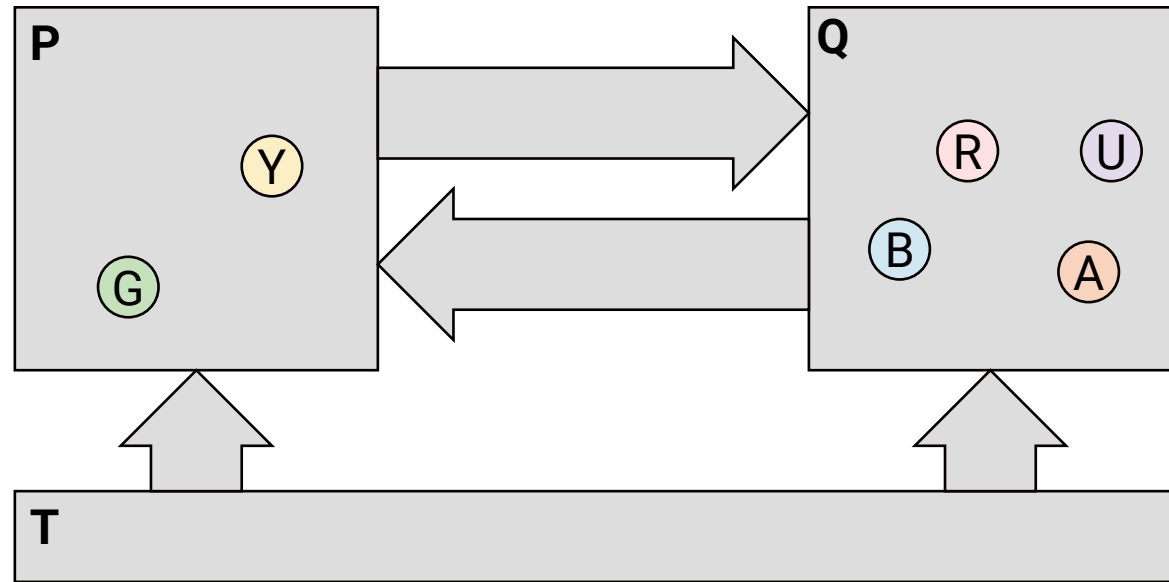
- Added a process T and just one channel $channel(T, P)$

Chandy-Lamport Puzzles #5

- Is this snapshot possible? And if so, how?

$P = \{ G, Y \}$
 $ch(P, Q) = \{ R \}$
 $Q = \{ B, A \}$
 $ch(Q, P) = \{ U \}$

 $T = \{ \}$
 $ch(T, P) = \{ \}$
 $ch(T, Q) = \{ \}$

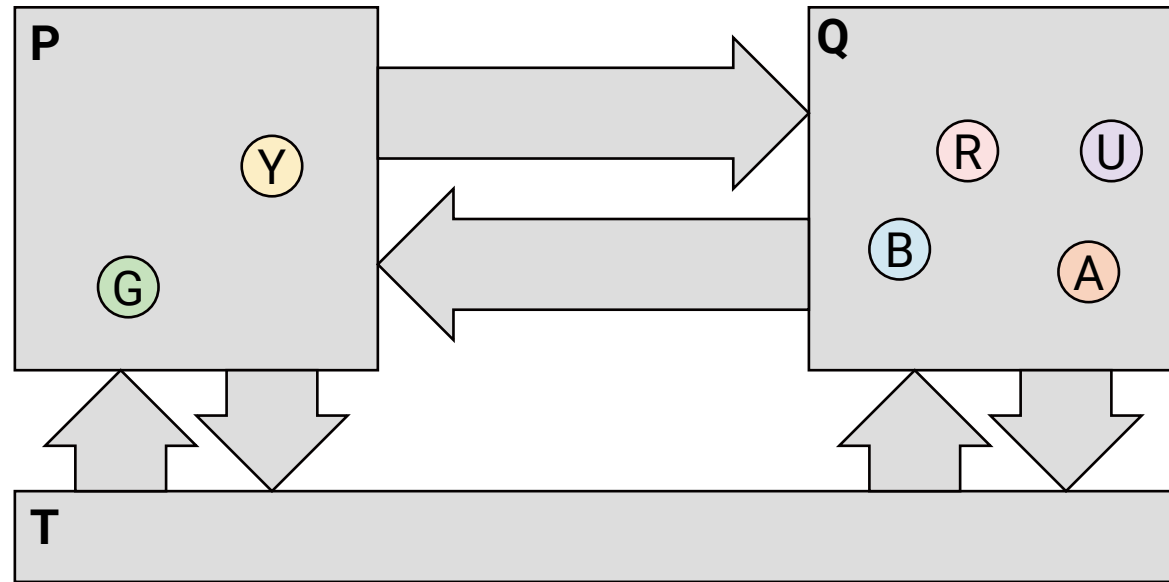


- Added another channel $channel(T, Q)$

Chandy-Lamport Puzzles #6

- Is this snapshot possible? And if so, how?

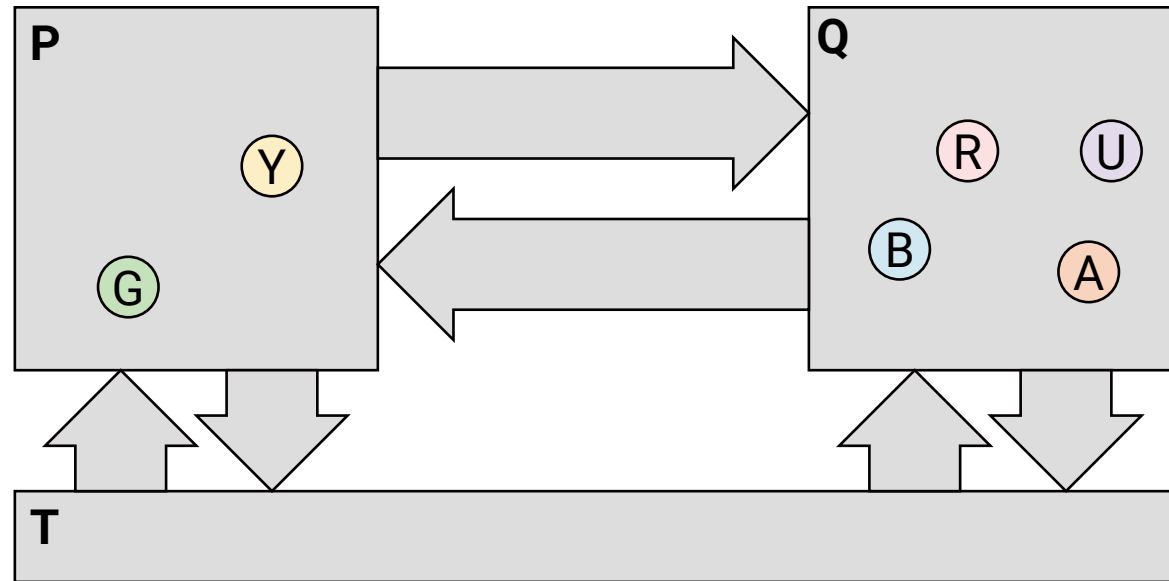
$P = \{ G, Y \}$
 $ch(P, Q) = \{ \}$
 $ch(P, T) = \{ \}$
 $Q = \{ B, A \}$
 $ch(Q, P) = \{ U \}$
 $ch(Q, T) = \{ R \}$
 $T = \{ \}$
 $ch(T, P) = \{ \}$
 $ch(T, Q) = \{ \}$



Chandy-Lamport Puzzles #7

- Is this snapshot possible? And if so, how?

$P = \{ G, Y \}$
 $ch(P, Q) = \{ \}$
 $ch(P, T) = \{ \}$
 $Q = \{ B \}$
 $ch(Q, P) = \{ U \}$
 $ch(Q, T) = \{ R \}$
 $T = \{ A \}$
 $ch(T, P) = \{ \}$
 $ch(T, Q) = \{ \}$



Terminating a Snapshot

- Distributed algorithm: No one process decides when it terminates
- Eventually, all processes have received a marker (and recorded their own state)
- All processes have received a marker on all the $N-1$ incoming channels (and recorded their states)
- Later, a central server can gather the local states to build a global snapshot



© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.